

Network Systems & Design COLL 400 Capstone Project

Adam Chisholm

Conner Wright

Elijah Gilliam

Zachary Klein

alchisholm@wm.edu

cjwright03@wm.edu

kmgilliam@wm.edu

zaklein@wm.edu

William & Mary

Williamsburg, Virginia, USA

Abstract

This project addresses the problem of classifying network traffic by its website of origin. The goal of this classification is to improve providers' ability to manage their networks based on users' activity. The traffic records we analyzed come from five websites – Backloggd, Github, Letterboxd, Reddit, and weather.com - and, based on nine characteristics, we classified these records using the Linear Regression, Logistic Regression, and Random Forest machine learning models. Comprehensive evaluation showed that the Random Forest model achieved the highest accuracy (0.72) and Macro F1 score (0.723), demonstrating its ability to handle complex relationships within the dataset. The project contributes to the field of network traffic classification by demonstrating the feasibility of integrating machine learning models into network traffic analysis, highlighting the importance of dataset diversity, and providing a road-map for future developments.

Keywords

Network Traffic Classification, Machine Learning, Classification Models

1 Introduction

Monitoring and classifying network traffic are critical tasks for maintaining efficient network management, security, and performance optimization [1]. The proliferation of diverse online services necessitates robust methods to categorize traffic patterns and accurately identify their sources [2]. This project aims to develop a machine learning model to classify network traffic by its originating website, leveraging the diversity of traffic patterns from different online services. Accurate classification of network traffic patterns has significant real-world implications. It facilitates network management by identifying traffic sources and patterns, helps security by detecting unusual behaviors, and optimizes performance by

ensuring balanced traffic flows. These capabilities are essential for managing and protecting modern networks.

We collected a dataset of traffic records from five popular websites: GitHub, Backloggd, Instagram, Letterboxd, and Weather. This dataset was split into training, and test sets in a 5:5 ratio. We employed three models for classification: Linear Regression [5], Logistic Regression [6], and Random Forest [4].

Comprehensive evaluation showed that the Random Forest model achieved the highest accuracy (0.72) and Macro F1 score (0.723), outperforming the other models. This demonstrates its ability to handle complex relationships within the dataset, facilitating accurate classification.

The project's contribution lies in its integration of machine learning models into network traffic analysis, demonstrating the feasibility and effectiveness of this approach in classifying traffic patterns by their originating website.

In summary, this project contributes to the field of network traffic classification, offering practical applications for network management, security, and performance optimization.

2 Proposed Method

In this section, we present our methodology for monitoring, analyzing, and classifying computer network traffic based on its originating website.

2.1 Data Preparation

To effectively monitor and analyze computer network traffic and classify it by website, we follow a systematic data preparation process as shown in Fig 1.

2.1.1 Tool Selection. In this project, we use Wireshark [2] (as shown in Fig 2), a highly regarded and versatile network protocol analyzer, to collect and analyze network traffic data. Wireshark offers a robust suite of features that allow us to capture, decode, and inspect network traffic at the packet level. Firstly, Wireshark provides detailed information about

Packet cou	Total packe	Average pa	Minimum	Maximum	Most comr	Avg. packe	Min. packe	Max. packe	Label
399	321262	805.1679	0	1460	1460	0.188927	0	14.28921	Backloggd
4895	4892917	999.5745	54	63414	1514	0.014789	-0.00183	8.507045	Backloggd
4509	4225882	937.2105	54	6982	1514	0.014176	-0.00325	4.840099	Backloggd
690	15702.6	22.7574	0	58.88807	1.683541	0.085469	0	18.4007	Backloggd
651	16311.09	25.05544	0	57.69191	13.61546	0.088757	0	7.063913	Backloggd
523	16519.25	31.58557	0	60.43214	11.93448	0.11577	0	8.555658	Backloggd
2260	66071.41	29.23514	0	65.22464	7.947095	0.028873	0	9.663236	Backloggd
938	33171.64	35.36423	0	63.14348	18.95352	0.067389	0	8.532003	Backloggd
1076	33318.79	30.96542	0	60.22023	55.98275	0.056019	0	10.08107	Backloggd
1725	48498.9	28.11531	0	63.67374	55.6642	0.036934	0	8.586525	Backloggd
855	252372	295.1719	0	64.01274	18.75878	0.074956	-1E-07	11.9999	Letterboxd
4565	3075097	673.6248	0	72.61562	1.777511	0.015911	-0.00045	5.107001	Letterboxd
1594	40727.3	25.55038	0	63.86929	1.552899	0.040094	0	12.53519	Letterboxd
972	20286.59	20.87098	0	46.73049	5.237568	0.048126	0	10.6903	Letterboxd
984	25948.87	26.3708	0	61.75431	17.46704	0.062822	0	19.44771	Letterboxd
1123	24250.74	21.59461	0	57.86993	21.58237	0.051577	0	18.79045	Letterboxd
1204	34495.09	28.6504	0	57.58438	31.46558	0.047867	0	14.75289	Letterboxd
1700	45649.51	26.85265	0	57.31608	15.10803	0.033735	0	7.022544	Letterboxd
1457	47860.14	32.84841	0	55.83169	13.77165	0.038346	0	6.769965	Letterboxd
1283	38872.6	30.29821	0	68.1949	0.082066	0.053194	-1.8E-05	17.19265	Letterboxd
837	930462	1111.663	0	54.1567	46.35796	0.064781	0	29.9803	GitHub
4134	2767007	669.3292	0	68.03958	29.93442	0.016463	-0.00067	5.576251	GitHub
676	396388	586.3728	0	62.80187	36.44262	0.092902	-1.6E-05	7.129898	GitHub
2127	77849.32	36.60052	0	106.1178	29.02002	0.049914	0	29.99983	GitHub
2028	76897.8	37.91805	0	66.64838	11.43937	0.03288	0	13.95637	GitHub

Figure 1: Network traffic features extracted from each capture

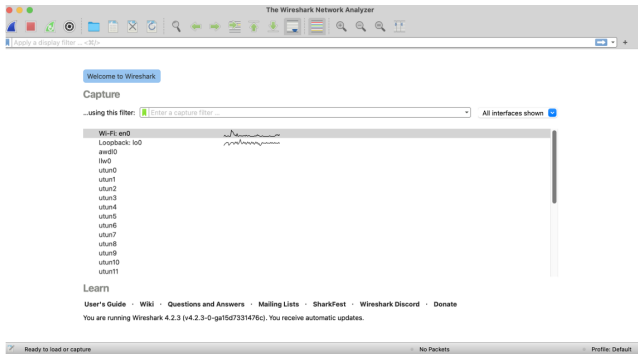


Figure 2: Wireshark's Interface.

individual packets, which includes information on protocols used, packet size, and content. This granular level of detail is crucial for our classification tasks, allowing us to discern specific characteristics and trends within network traffic. Secondly, the tool supports a wide variety of protocols and formats, making it an ideal choice for handling diverse network environments. This flexibility ensures we can handle various types of traffic effectively. Thirdly, Wireshark's graphical user interface offers an intuitive way to explore and interpret network traffic data. The interface provides features such as real-time packet capture, filtering options, and detailed views of packet content, streamlining our analysis process.

2.1.2 Data Collection. We initiate the data collection process by opening Wireshark, a network protocol analyzer, and configuring filters to capture traffic from specific web sites: GitHub, LetterBoxd, Backloggd, Reddit, and Weather. This diverse selection of sites generates unique traffic patterns

representative of popular online services. We visit each web site, navigating its pages for a minimum of one minute to gather traffic data. This browsing session results in a single data record containing all packets captured during that time. We repeat this process for each of the five websites, creating a comprehensive raw dataset. The variety of captured packets ensures a representative sample of each website's traffic.

2.1.3 Feature Engineering. After collecting a sufficient number of data records, each consisting of multiple packets, we proceed to feature engineering. The raw data cannot be directly used for training, thus we extract eight key features from each record:

- **Packet Count:** This feature indicates the total number of packets in each record, reflecting the overall communication volume. It's a key indicator of the traffic intensity and can reveal insights into the website's usage patterns.
- **Total Length:** This feature represents the cumulative size of all packets in a record, encompassing both headers and payloads. It provides a measure of the data throughput and is crucial for understanding the traffic load and capacity requirements.
- **Packet Length Variance:** This feature corresponds to the sample variance of all packet lengths within a capture. It quantifies the diversity of packet lengths in the record.
- **Average Packet Interval:** This feature calculates the mean time difference between consecutive packets, of inferring insights into the temporal distribution of traffic. A higher average interval might indicate sporadic traffic, while a lower interval suggests a steady flow.
- **Maximum Packet Interval:** This feature captures the longest time gap between two packets in a record, helping identify periods of inactivity or delays. This can reflect the responsiveness of the website or the nature of its traffic.
- **Minimum Packet Interval:** This feature denotes the shortest time gap between two packets, indicating the speed at which data is transferred during active periods. It reflects the network's latency and the website's responsiveness.
- **Packet Interval Variance:** This feature corresponds to the sample variance of all time intervals between packets. It quantifies the diversity of intervals that elapse between two packets.
- **Average Packet Length:** This feature calculates the mean

size of all packets in a record, offering insights into the average data volume transferred per packet. It can indicate the nature of the website's content (e.g., text heavy vs. media-heavy).

- **Maximum Packet Length:** This feature indicates the size of the largest packet in a record, reflecting the maximum data transfer capacity in a single packet. It can reveal the presence of large files or media content.

- **Minimum Packet Length:** This feature captures the size of the smallest packet in a record, reflecting minimal data transfer requirements. It can indicate small, frequent communications or requests.

2.2 Employed Method

For our project on monitoring and analyzing network traffic, particularly from popular internet websites, and classifying the originating website, we employ the Random Forest algorithm [4]. This algorithm is integral to our solution due to its flexibility, robustness, and strong performance in handling complex classification tasks.

2.2.1 Random Forest. Random Forest is an ensemble learning method primarily used for classification and regression tasks. It operates by constructing multiple decision trees during the training phase and aggregating their outputs to make a final decision. The process of Random Forest is:

- **Building the Forest:** During training, multiple decision trees are generated. Each tree is built from a bootstrapped sample of the training dataset. In each tree, a subset of features is randomly chosen to split at each node, which helps introduce diversity among the trees.

- **Voting Mechanism:** In classification tasks, each tree votes on the class label for a given input. The class that receives the most votes from the trees in the forest becomes the final prediction.

- **Robustness and Stability:** This ensemble approach makes Random Forest resistant to overfitting, a common issue in single decision trees, and provides a stable and reliable model even in the face of noisy data or variations in feature sets. The benefits of Random Forest includes:

- **Handling Complex Features:** The task of classifying network traffic by its originating website involves handling diverse and complex features, such as protocols and packet lengths. Random Forest's decision trees can efficiently capture interactions between these features, producing accurate

classifications.

- **Interpretability:** While individual decision trees may be challenging to interpret, Random Forest offers useful insights into feature importance. This capability is valuable for understanding which features, such as protocol type or packet length, contribute most to distinguishing traffic from different websites.

2.2.2 Baselines. We compared two baselines: • Linear Regression [5] serves as a baseline model for continuous numerical predictions due to its simplicity, interpretability, and straightforward implementation. It provides a linear relationship between the dependent variable and one or more independent variables, making it easy to understand and communicate results. The Ordinary Least Squares estimation minimizes the sum of squared errors, offering a quick way to generate initial predictions. Its assumptions, though restrictive, provide a useful framework for initial model evaluation, making it a reliable starting point for comparing more complex models in regression tasks. • Logistic Regression [6] is chosen as a baseline model for binary and categorical classification tasks due to its simplicity, interpretability, and effectiveness in probability based predictions. Its logistic function directly maps features to probabilities, offering clear insight into the likelihood of a particular outcome. Maximum Likelihood Estimation provides a robust way to fit the model to data, and its assumptions lay the groundwork for initial model validation. This makes it a valuable baseline for comparing more complex models in classification tasks.

3 Evaluation

In this section, we perform an in-depth evaluation of our models and their performance on the dataset.

3.1 Datasets

After processing the collected data, we obtained the dataset. The train and test sets were split with a ratio of 5:5.

3.2 Evaluation Metrics

We use two primary metrics to evaluate the models: Accuracy and Macro F1 score. These metrics provide a comprehensive view of performance across all classes.

The **Accuracy** is defined as:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$$

The **Precision** is defined as:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Model	Accuracy	Macro F1 Score
Linear Regression	0.4	0.348
Logistic Regression	0.52	0.52
Random Forest	0.72	0.723

Table 1: Caption

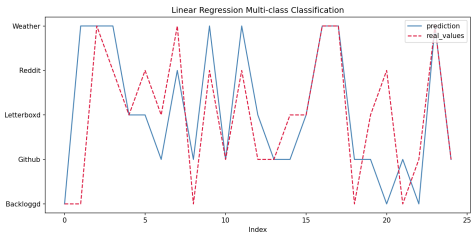


Figure 3: Linear Regression Model

Precision indicates how many of the predicted positive samples are actually positive. The **Recall** is defined as:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Recall reflects how many of the actual positive samples are correctly identified as positive. The **Macro F1 Score** is calculated by taking the harmonic mean of precision and recall for each class, and then averaging these scores across all classes:

$$\text{Macro F1} = \frac{1}{n} \sum_{i=1}^n \frac{2 \times \text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

3.3 Evaluation Results

The results in Table 1 show that the Random Forest model achieves the highest accuracy (0.72) and Macro F1 score (0.723), outperforming both Linear and Logistic Regression models. This indicates that the Random Forest model is more effective at capturing complex relationships in the data.

3.4 Visualization Results

To evaluate how each model’s predictions align with actual values, we visualize the predictions and real values for each model: Linear Regression (Fig 4), Logistic Regression (Fig 5), and Random Forest (Fig 6). The results show that the Random Forest model exhibits the closest alignment between predictions and actual values, reinforcing its superior performance observed in the quantitative metrics.

4 Discussion & Future Work

4.1 Interpretation of Results

Three models were tested: Linear Regression, Logistic Regression, and Random Forest. The results indicate that Random

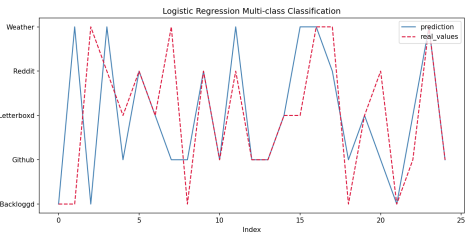


Figure 4: Logistic Regression Model

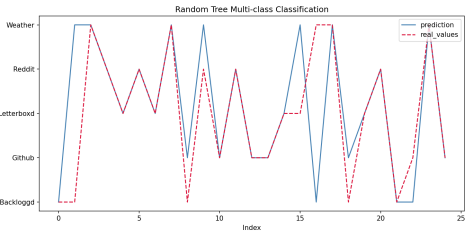


Figure 5: Random Forest Model

Forest outperformed both Linear and Logistic Regression in terms of both Accuracy and Macro F1 score. Specifically, Random Forest achieved an accuracy of 0.72 and a Macro F1 score of 0.723, compared to Logistic Regression’s accuracy of 0.52 and Macro F1 score of 0.52. This performance gap highlights Random Forest’s ability to capture complex relationships in the data, making it particularly suitable for network traffic classification, where patterns can vary significantly. The visualizations further reinforce the quantitative findings. The figures show that the Random Forest model’s predictions align closely with the actual values, providing a visual affirmation of its superior performance. The evaluation and comparison of these models contribute significantly to the problem at hand: monitoring and classifying network traffic. The high accuracy and precision of the Random Forest model allow for effective classification of diverse traffic patterns, supporting efficient network management and optimization. This is particularly relevant for identifying unusual or potentially harmful traffic patterns, which is crucial for network security.

4.2 Future Work

Despite the strong performance of Random Forest, the project faced some challenges:

- **Dataset Size:** The relatively small dataset size might limit the model’s ability to generalize to broader traffic patterns. This may also impact the model’s performance when encountering new traffic types or variations in existing ones.

- **Handling Real-Time Data:** The current methodology captures data from static browsing sessions, but real world network environments often involve rapidly accumulating and dynamic traffic. This necessitates handling real-time data streams effectively.

- **Model Complexity:** While the Random Forest model performs well, its ensemble nature and depth can introduce interpretability challenges, making it harder to understand which specific features drive its decisions. To address these challenges and improve the project further, we propose the following directions:

- **Dataset Expansion:** Expanding the dataset to include more websites and traffic records, as well as capturing traffic patterns over longer durations, will enhance the model's training, making it more generalizable and robust. Advanced Models: Exploring other machine learning models, such as deep learning or ensemble approaches, could further improve classification accuracy by capturing more complex traffic relationships.

- **Real-Time Processing:** Enhancing the model to handle real-time data streams effectively will improve its practical utility in dynamic network environments. This could involve integration with streaming analytics platforms or developing custom solutions for real-time traffic capture and analysis.

- **Practical Applications:** Future research should also focus on practical applications, integrating the model into real-world systems for network monitoring, management, and security, demonstrating its value beyond academic contexts. This could involve collaborating with

dataset size and diversity in model training, highlighting the need for expansion in future work. 3) The project's visualizations reveal how each model's predictions align with actual values, providing further insights into their effectiveness. This project serves as a foundation for future developments in network traffic analysis, with practical implications for real-world systems and applications. By integrating machine learning models into network monitoring, we can enhance performance, security, and management.

References

- 1 K. Kumar, M. Punia, and Vandana. 2023. Network traffic classification techniques: A survey. In Proceedings of the 2023 Seventh International Conference on Image Information Processing (ICIIP). IEEE, Solan, India, 205–211. <https://doi.org/10.1109/ICIIP61524.2023.10537766>
- 2 Wireshark Foundation. 2024. Wireshark. <https://www.wireshark.org/> Available: <https://www.wireshark.org/>
- 3 J. Hao and T. K. Ho. 2019. Machine learning made easy: A review of “Scikit-learn” package in Python programming language. *Journal of Educational and Behavioral Statistics* 44, 3 (2019), 348–361. <http://www.jstor.org/stable/4433333>
- 4 Scikit-learn Developers. n.d. `sklearn.ensemble.RandomForestClassifier`. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- 5 GeeksforGeeks. 2026. Implementing Linear Regression from Scratch using Python. <https://www.geeksforgeeks.org/machine-learning/linear-regression-python-implementation/>
- 6 GeeksforGeeks. 2026. Logistic Regression using Python. <https://www.geeksforgeeks.org/machine-learning/ml-logistic-regression-using-python/>
- 7 GeeksforGeeks. 2025. How to Optimize Linear Regression Performance. <https://www.geeksforgeeks.org/machine-learning/how-to-optimize-logistic-regression-performance/>

5 Conclusion

In this project, we developed and evaluated a model for monitoring and classifying network traffic by its originating web site. Using a dataset collected from diverse websites, we employed three models: Linear Regression, Logistic Regression, and Random Forest. Through comprehensive evaluation, the Random Forest model was found to outperform the others, achieving the highest accuracy and Macro F1 score. This work demonstrates the feasibility of machine learning models, particularly Random Forest, for handling complex relationships in network traffic data. Accurate classification of network traffic patterns contributes significantly to efficient network management, security, and performance optimization. The main insights include: 1) The Random Forest model's ensemble approach provides robustness and superior performance, particularly in handling diverse traffic patterns. 2) The project's results indicate the importance of